

**What is Software Process:** A software process is a set of related activities that lead to the production of a software product.

Software Process involves: Specification, Design & Implementation, Validation, Evolution.

## Process Model

Defines a distinct set of activities, actions, tasks, milestones, and work products that are required to engineer high-quality software. The activities may be linear, incremental, or evolutionary

## Software Process Models:

### 1. Waterfall Model

**Analogy:** Building a house. You must complete the foundation perfectly before you build the walls, and the walls must be perfect before you put them on the roof. There's no going back once a phase is complete.

**In Detail:**

The Waterfall model is a linear, sequential approach where each phase must be completed fully before the next one can begin. It's highly structured and document driven.

**Plan and schedule all of the process activities before starting work on them**

**Phases (in strict order):**

1. Requirements Gathering: Document all customer requirements in detail. This is the most crucial phase. Once signed off, they are frozen.
2. System Design: Architects and designers create technical blueprints for the system based on the requirements.
3. Implementation (Coding): Developers write the code according to the design documents.
4. Testing: Testers verify the software against the requirements defined in phase 1. Bugs are fixed.
5. Deployment: The finished product is delivered to the customer.
6. Maintenance: Fixing any issues found in the live environment and releasing updates.

**When to use it:**

- When requirements are very well understood, clear, and unlikely to change.
- For short, simple projects.
- For projects with strict regulatory requirements that demand extensive documentation (e.g., medical, military systems).

#### Advantages:

- Simple and easy to understand and manage.
- Clear milestones and deliverables.
- Extensive documentation is produced.

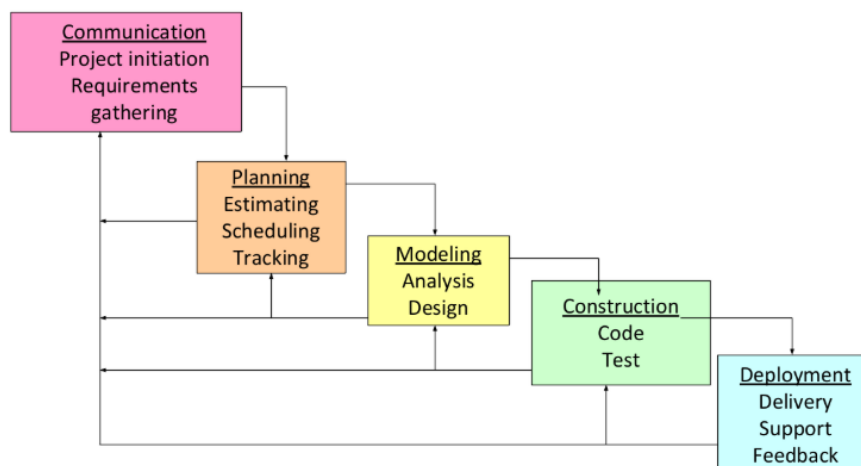
#### Disadvantages:

- Inflexible. It's very difficult and expensive to go back and change something from a previous phase.
- Working software is produced very late in the life cycle (only after testing).
- High risk and uncertainty. The customer doesn't see the product until the very end.
  - **Software requirements change**, hard to sign-off on a SRS.
  - **Early commitment**. Changes at the end, large impact.
  - **Feedback** is needed to understand a phase. E.g. implementation is needed to understand some design.
  - **Difficult to estimate** time and cost for the phases.
  - **Handling risks** are not part of the model. Pushes the risks forward.
  - Software **"is not"** developed in such a way. It evolves when problems are more understood.
  - **Doesn't support iteration**, so changes can cause confusion
  - **Difficult for customers to state all requirements** explicitly and up front
  - **Requires customer patience** because a working version of the program doesn't occur until the final phase
  - **Problems** can be somewhat **alleviated** in the model through the addition of feedback loops

Software Engineering

17

## Waterfall Model with Feedback (Diagram)



## 2. V-Model (Verification and Validation Model)

**The Analogy:** The Waterfall model's more meticulous cousin. For every development phase, there is a corresponding testing phase.

### In Detail:

The V-Model is an extension of the Waterfall model that emphasizes quality assurance by linking each development phase directly to a testing phase. It's often used in mission-critical systems.

### The "V" Shape:

- The left side of the "V" represents the decomposition of requirements and creation of system specifications.
- The right side of the "V" represents integration and testing of components.

### Corresponding Phases:

- Requirements Analysis <-> Acceptance Testing (Does the system do what the user needs?)
- System Design <-> System Testing (Does the integrated system meet the specs?)
- Architectural Design <-> Integration Testing (Do the modules work together correctly?)
- Module Design <-> Unit Testing (Do individual units of code work correctly?)

### When to use it:

- Similar to Waterfall: requirements are fixed, well-defined, and clear.
- Where high reliability and strict quality standards are required (e.g., aerospace, medical devices).

### Advantages:

- Higher chance of success due to tough testing plans being created early.
- Defects are found much earlier than in the classic Waterfall model.
- Avoids the downward flow of defects.
- It defines **tangible phases** of the process, and proposes a logical sequence in which these phases should be approached
- It also defines **logical relationships** between the phases
- It demands that **testing** documentation is written as soon as possible
- It gives **equal weight** to development and testing
- It provides a **simple and easy to follow map** of the software development process

### Disadvantages:

- Shares all the disadvantages of the Waterfall model (inflexible, late delivery).
- Very rigid and can be more costly to implement due to the high level of testing.
- It is too **simple to accurately** reflect the software development process
- It is **inflexible**; it has no ability to respond to **change**
- It produces **inefficient testing** methodologies



## Phased Development

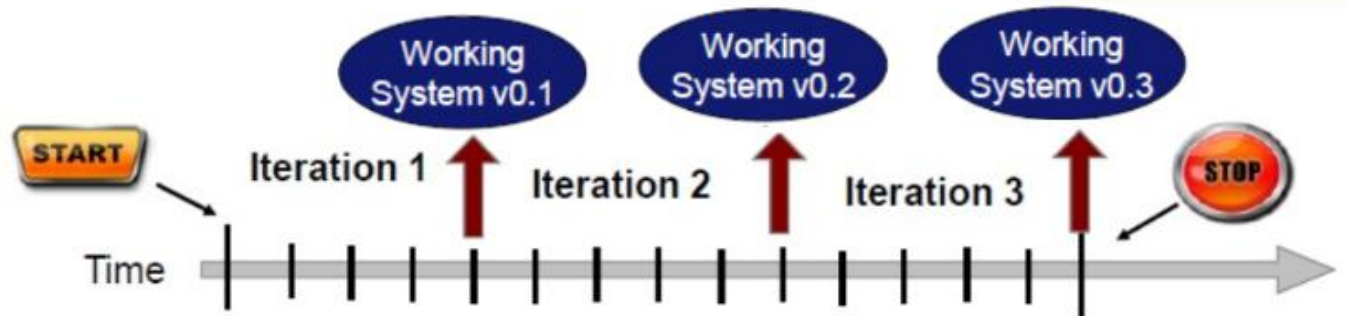
- Design a system so it can be delivered in pieces, letting users have some functionality while the rest is under development
- There are usually two or more systems in parallel:
  - a) The **operational** or **production** system in use by customers
  - b) The **development system** which will replace the current release

### 3. Iterative Model

**Analogy:** Sculpting a statue. You start with a rough block of stone (a basic version) and repeatedly refine it, adding more detail with each pass.

#### In Detail:

Instead of starting with full, known requirements, you start with a small set of requirements, implement and test them, then iteratively add more features in repeated cycles. Each iteration is a "mini-project" that produces a working, albeit incomplete, version of the software.



#### Key Steps per Iteration:

1. Planning (Requirements for this iteration)
2. Design & Implementation
3. Testing & Evaluation
4. Review (Plan the next iteration based on feedback)

#### When to use it:

- For large projects where the major requirements are understood but details may evolve.
- When the core features are needed first, and secondary features can be added later.
- When new technology is being used, the team is learning along the way.

#### Advantages:

- Working software is delivered early and frequently.
- Easier to accommodate changing requirements.
- Risks are identified and mitigated early (e.g., technical challenges in early iterations).

#### Disadvantages:

- Requires more management attention.
- Not suitable for small projects.
- Architectural problems can arise if not planned for in the initial iteration.

## 4. Incremental Model

**The Analogy:** Building a Lego set. You build the baseplate, then the first section (e.g., the engine), then the next section (the cockpit), and so on. Each section is a fully functional part of the whole.

#### In Detail:

The software is broken down into several standalone modules (increments). Each increment is developed

separately using a mini-Waterfall model. Each module adds some complete, usable functionality to the previous one.

## Incremental Development

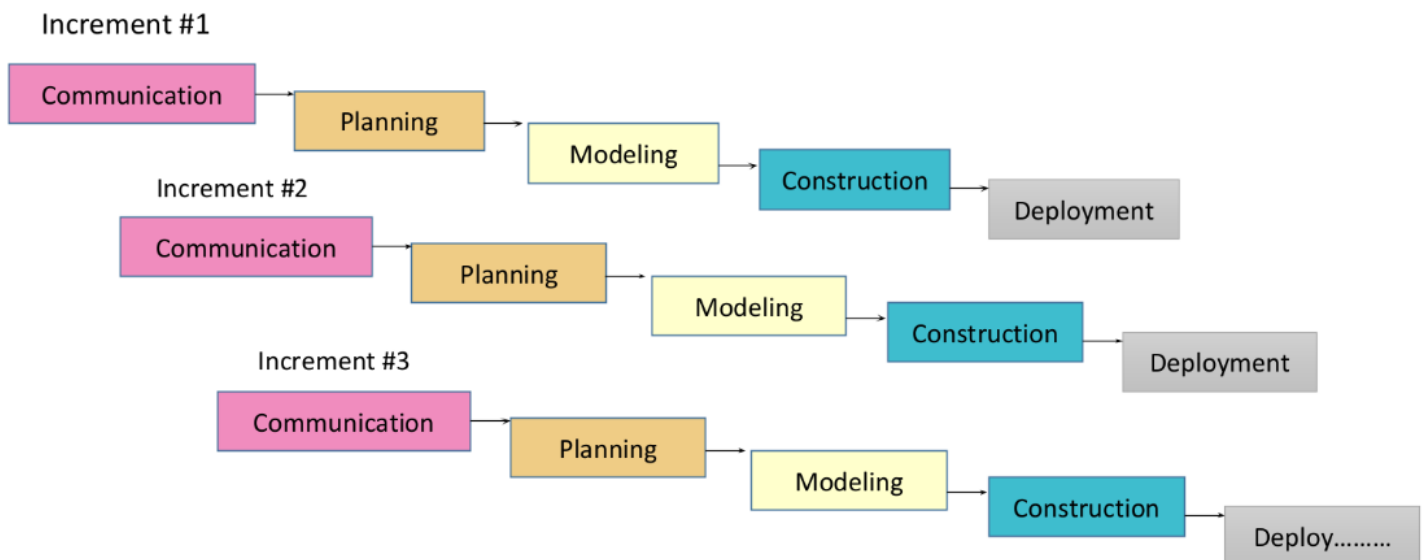
Add a new "part" at each increment



**Crucial Note:** People often confuse Iterative and Incremental. Here's the key difference:

- **Iterative:** You build the *entire system* in repeated passes, adding more detail each time. (First iteration: a basic car with just wheels and a frame. Second iteration: add a basic engine. Third iteration: add seats).
- **Incremental:** You build *separate pieces* of the system one by one and then combine them. (First increment: build a fully functional engine module. Second increment: build a fully functional cockpit module. Then integrate them).

In practice, most modern models (like Agile) are Iterative *and* Incremental.



### When to use it:

- When requirements are well understood.
- When there is a need to get basic functionality to the market early.
- When teams are not available for the entire project (different teams can work on different increments).
- When the project has a long timeline and priorities may shift.

### Advantages:

- Customers can use core features early without waiting for the full product.

- Lower risk of overall project failure.
  - Easier to test and debug smaller increments.
- ✧ The cost of accommodating changing customer requirements is reduced.
    - The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
  - ✧ It is easier to get customer feedback on the development work that has been done.
    - Customers can comment on demonstrations of the software and see how much has been implemented.
  - ✧ More rapid delivery and deployment of useful software to the customer is possible.
    - Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

#### **Disadvantages:**

- Requires a very clear and modular design from the start.
  - Total cost is often higher than a single-pass model like Waterfall.
  - Integrating increments can be problematic if not planned perfectly.
- ✧ The process is not visible.
    - Managers need regular deliverables to measure progress. If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.
  - ✧ System structure tends to degrade as new increments are added.
    - Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure. Incorporating further software changes becomes increasingly difficult and costly.

## **5. Prototyping Model**

**The Analogy:** A car manufacturer creating a concept car. It's not meant to be driven daily, but to test ideas, generate excitement, and get feedback before building the real thing.

#### **In Detail:**

This model involves creating a mock-up version of the software (a prototype) specifically to understand requirements and gather feedback early.

#### **Types of Prototyping:**



- **Throwaway / Rapid Prototyping:** A quick, dirty prototype is built to clarify requirements. It is then discarded, and the real system is built properly from scratch. (Like a sketch on a napkin).
- **Evolutionary Prototyping:** The prototype is not discarded. Instead, it is continuously refined and improved based on feedback until it eventually becomes the final system. (Like a living document). It helps in building the product incrementally as requirements become clear over time.
- **Horizontal / Behavioral Prototyping:** Prototypes the user interface only (the "wide" part of the system). It shows the screens and navigation but has little to no real functionality. Good for getting UI/UX feedback.
- **Vertical / Structural Prototyping:** Prototypes a single, specific feature in full depth (a "slice" of the system). It has a real UI and a real backend for that one feature. Good for proving a technically risky concept.

- The throwaway prototype is most **appropriate** when the team faces uncertainty, ambiguity, incompleteness, or vagueness in the requirements
- A prototype that helps users and developers visualize how the requirements might be implemented can reveal gaps in the requirements

#### When to use it:

- When requirements are ambiguous or poorly understood.
- When the user interface is a critical part of the project.
- To validate technical feasibility.

#### Advantages:

- Reduces ambiguity and misunderstanding of requirements.
  - Very useful for getting user engagement and feedback early.
  - Help in revealing missing functionality.
- Suitable for **large systems** for which there is no manual process to define the requirements
  - Much **feedback** from customer
  - **Quality** of software is good



### Disadvantages:

- It can lead to **scope creep** if users see the prototype and keep asking for more. The uncontrolled expansion of a project's scope, involving the addition of extra features, requirements, or work beyond what was initially planned and approved, often leading to increased costs, longer timelines, and potentially jeopardized project objectives.
  - Developers might make compromises in the prototype ("quick and dirty") that end up in the final product if using an evolutionary approach.
  - The cost of building a throwaway prototype can be high.
- It may be impossible to tune the prototype to meet **non- functional** requirements
  - Lot of **time** at customer
  - The **changes** made during prototype development will probably have **degraded** the system structure

## 6. Spiral Model

The Analogy: Planning a very risky trip. You plan a little, go a short distance, evaluate the risk, check your supplies, and then plan the next part of the journey.

### In Detail:

The Spiral Model is a risk-driven process model that combines the iterative nature of prototyping with the controlled, systematic aspects of the Waterfall model. Each "loop" or "spiral" around the four quadrants represents a phase.

### The Four Quadrants of each Spiral:

1. Objective Setting (Plan): Define goals, alternatives, and constraints.
2. Risk Analysis and Resolution (Evaluate): This is the key quadrant. Identify risks (technical, managerial, etc.) and plan how to resolve them (e.g., through prototyping, research).
3. Development and Testing (Execute): Develop and test the solution for this iteration using the best-suited model (e.g., Waterfall, prototyping).
4. Planning the Next Iteration (Review): Plan the next phase based on feedback and risk analysis.

### When to use it:

- For large, expensive, complex, and high-risk projects.

- When requirements are unclear or complex.
- When significant changes are expected during development.

**Advantages:**

- High amount of risk analysis makes it great for avoiding major failures.
- Very flexible, as requirements can change between spirals.
- Software is produced early in the life cycle.

**Disadvantages:**

- Can be very complex and expensive to manage.
- Requires expertise in risk assessment.
- Not suitable for small or low-risk projects.